

Como Python pode ser utilizado para carregar, limpar, transformar e analisar dados provenientes de uma planilha?

Thoughts

O processamento de planilhas com Python combina a biblioteca Pandas — especializada na manipulação de dados tabulares organizados em DataFrames — e o OpenPyXL, que atua na leitura e gravação nativa de arquivos no formato Excel (.xlsx).

O fluxo completo para carregar, limpar, transformar e analisar uma planilha divide-se nas quatro etapas detalhadas abaixo:

1. Carregamento dos Dados (Leitura)

O processo inicia-se com a leitura da planilha do disco para a memória. A função `pd.read_excel()` do Pandas lê o arquivo .xlsx e constrói uma estrutura bidimensional denominada DataFrame, utilizando o OpenPyXL como motor de leitura.

```
import pandas as pd

# Carrega a planilha Excel para um DataFrame
dados = pd.read_excel("vendas.xlsx")
```

2. Limpeza e Validação de Qualidade dos Dados

Antes de realizar análises ou calcular indicadores, é necessário tratar inconformidades como tipos incorretos de dados e valores nulos.

Conversão de Tipos: A função `pd.to_numeric(..., errors="coerce")` força a conversão de uma coluna para o tipo numérico; caso encontre textos corrompidos ou entradas inválidas, ela os transforma automaticamente em valores nulos (NaN).

Tratamento de Nulos e Inconsistências: Métodos como `.dropna()` permitem eliminar registros com células vazias, garantindo a integridade dos cálculos subsequentes.

```
# Converte a coluna 'Valor' para número, transformando textos inválidos em NaN
dados["Valor"] = pd.to_numeric(dados["Valor"], errors="coerce")

# Elimina as linhas que ficaram com o valor ausente/nulo
dados_limpos = dados.dropna(subset=["Valor"])
```

3. Transformação e Análise dos Dados

Com os dados validados, são aplicadas as regras de negócio para isolar e consolidar as informações necessárias.

Filtragem Lógica: Seleciona um subconjunto específico de linhas a partir de condições condicionais (por exemplo, filtrar apenas transações com status "Pendente" ou "Concluído").

Agrupamento e Agregação (Split-Apply-Combine): O método `.groupby()` divide a tabela em grupos categorizados (como filiais ou produtos). Em seguida, aplicam-se funções estatísticas de agregação, como a soma `.sum()`, para consolidar os resultados por grupo.

```
# Filtragem: isola apenas os registros com Status "Pendente"
pendentes = dados_limpos[dados_limpos["Status"] == "Pendente"]

# Agrupamento e Agregação: agrupa por 'Unidade' e soma a coluna 'Valor'
resumo_unidade = (
    pendentes
    .groupby("Unidade", as_index=False)["Valor"]
    .sum()
    .rename(columns={"Valor": "Valor_Total_Pendente"})
)
```

4. Exportação dos Resultados

Após a transformação e análise, o DataFrame consolidado pode ser salvo em um novo arquivo Excel utilizando o método `.to_excel()`, mantendo a planilha original preservada.

```
# Exporta o resultado para um novo arquivo Excel
resumo_unidade.to_excel("relatorio_resumo.xlsx", index=False)
```

Exemplo Prático Integrado e Explicação Passo a Passo

Abaixo está o script completo unindo todas as etapas do processo:

```
import pandas as pd

# 1. Carregamento da planilha
dados = pd.read_excel("pagamentos.xlsx")

# 2. Limpeza: coerção da coluna Valor para tipo numérico
dados["Valor"] = pd.to_numeric(dados["Valor"], errors="coerce")

# 3. Transformação: filtragem dos registros pendentes
pendentes = dados[dados["Status"] == "Pendente"]

# 4. Análise: agrupamento por Unidade e soma do Valor
resumo = (
    pendentes
    .groupby("Unidade", as_index=False)["Valor"]
    .sum()
    .rename(columns={"Valor": "Valor_Pendente"})
)

# 5. Exportação do novo arquivo Excel
resumo.to_excel("relatorio_pendencias.xlsx", index=False)

print("Relatório gerado com sucesso!")
```

Explicação passo a passo do código:

`import pandas as pd`: Importa a biblioteca Pandas atribuindo o apelido padrão `pd`.

`pd.read_excel("pagamentos.xlsx")`: Lê a planilha de origem e armazena os registros no DataFrame `dados`.

`pd.to_numeric(dados["Valor"], errors="coerce")`: Valida a coluna "Valor", garantindo que todas as entradas sejam tratadas como números e convertendo eventuais falhas em NaN.

`dados[dados["Status"] == "Pendente"]`: Executa a filtragem condicional, mantendo apenas as linhas onde a coluna "Status" seja exatamente igual a "Pendente".

`.groupby("Unidade", as_index=False)`: Separa os registros filtrados em grupos baseados nos nomes das filiais contidos na coluna "Unidade".

`["Valor"].sum()`: Aplica a soma aos valores numéricos de cada filial agrupada.

`.rename(columns={"Valor": "Valor_Pendente"})`: Renomeia a coluna resultante para deixar o cabeçalho do relatório claro.

`.to_excel("relatorio_pendencias.xlsx", index=False)`: Salva a tabela consolidada em uma nova planilha .xlsx com o suporte do OpenPyXL, utilizando `index=False` para ocultar a numeração automática de linhas do Pandas.